

Table 1: The differences between private and public blockchains

Feature	Ethereum (Public)	Hyperledger Fabric (Private)
Access	Open to everyone	Permissioned access via X.509 certs
Consensus	Proof of Stake (PoS)	Practical Byzantine Fault Tolerance (PBFT)
Latency	Average of 720 milliseconds (ms)	Average of 45 milliseconds (ms)
Throughput	Approximately 15 to 30 transactions per second (TPS)	Approximately 2000 plus transactions per second (TPS)
Visibility	Data is publicly accessible to everyone	Limited visibility through private channels
Smart contracts	Developed using Solidity programming language	Implemented as chaincode using Go or Java programming language
Suitable for UAVs	No	Yes

UAV swarm architecture

Figure 1 explains how a group of UAVs work together. It also depicts how swarms collect data and encrypt it before sending it through a smart network that can detect attacks in real time. A blockchain system ensures everything is verified; it makes sure everything is trusted and safely recorded without tampering.

The technology stack we used is: MATLAB for simulation, Python for data processing, Hyperledger Fabric with Go programming language for blockchain and chaincode implementation, NSL-KDD dataset for IDS training, and AES-256 and ECC P-256 for cryptography.

Figure 2 depicts the real-time UAV swarm security dashboard showing live mesh topology. It clearly gives the details about the Hyperledger Fabric block log and AES-256 + ECC P-256 encryption metrics.

In Hyperledger Fabric, chaincodes are nothing but the smart contracts that run the swarms. Every drone must register itself before transmitting even a single byte of information. The chaincode given below validates the MSP certificate and also writes the UAV identity permanently to the blockchain ledger so that it becomes immutable:

```
type UAVIdentity struct {
    UAVID      string `json:"uav_id"`
    CertHash   string `json:"cert_hash"`
    SwarmID    string `json:"swarm_id"`
    Status     string `json:"status"` // ACTIVE |
    SUSPICIOUS | REVOKED
    RegisteredAt string `json:"registered_at"`
}

func (r *UAVRegistry) RegisterUAV(ctx contractapi.
TransactionContextInterface,
    uavID, certHash, swarmID, timestamp string) error {
    existing, _ := ctx.GetStub().GetState(uavID)
```

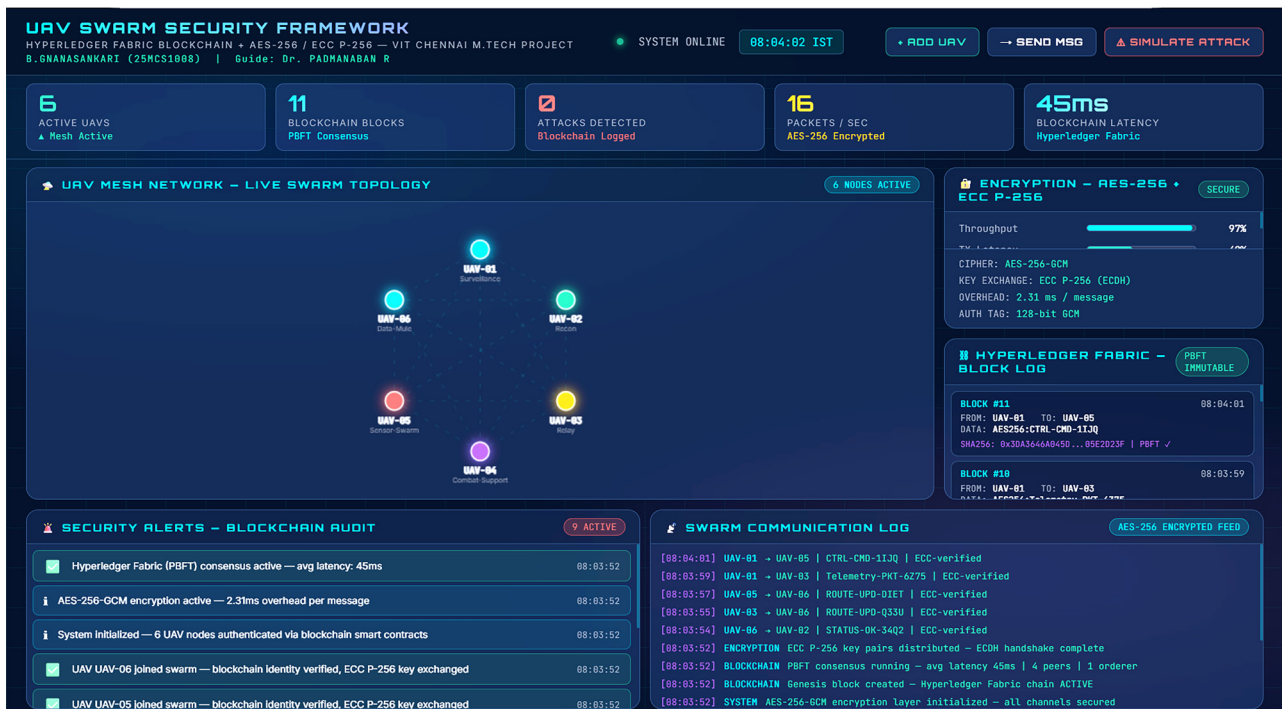


Figure 2: UAV swarm security dashboard